
CASE STUDY / 2026

Synthetic Data Engine

Rule-based daily KPI generator powering a self-refreshing investor data room.

Python • pandas • Google Sheets API — Data generation & warehouse-as-spreadsheet

PREPARED BY

Mudasir Shafiq

Full-stack & data engineer · chmudasir.fr@gmail.com

CONTENTS

Table of Contents

01	Project Overview	Page 3
02	Pipeline Architecture	Page 4
03	Rules & Dependency Chain	Page 5
04	Validator Checks	Page 6
05	Technology Stack	Page 7
06	The Challenge	Page 8
07	The Approach	Page 9
08	Code Spotlight	Page 10
09	Outcome & Engagement	Page 11

This document presents an in-depth look at the Synthetic Data Engine — the rule layer, the disaggregation logic, the validator that audits every synthetic row, and the idempotent Sheets push that keeps the live data room fresh. Every diagram, code snippet, and number is grounded directly in the project source.

SECTION 01

Project Overview

What the engine does, what it produces, and why it never breaks a dashboard.

The **Synthetic Data Engine** is a Python toolkit that takes a client's real monthly KPI history and turns it into a continuously updating *daily* stream of synthetic-but-realistic rows, pushed straight into a Google Sheet that powers investor dashboards. The engine is designed around one principle: every row it produces must obey the same set of explicit business rules that the original data did — cumulative metrics never decrease, dependent metrics never invert, engagement is ordered correctly, revenue is calculated transparently — so the dashboards never break and stakeholders never lose trust.

The platform ships as three coordinated Python modules plus a cron-safe scheduler. A one-shot *historical_data_daily_generator* splits each monthly total into daily values using Dirichlet weights with capped maximums, asserting that the monthly sums round-trip exactly. A daily *generator* appends new rows that follow the ranges encoded in *config.py*. A strict *validator* re-checks every synthetic row against the full rule set. A separate Sheets-upsert script then writes the result with a hard guard against re-appending existing dates.

The output is a clean, growing time-series that behaves like a real product over time — and is safe to show to investors. Every synthetic row is flagged *is_synthetic=True* so a future analyst can always tell history apart from the generator's output.

22

Months of history

626

Daily rows

12+

Validator checks

idempotent

Sheets upsert

SECTION 02

Pipeline Architecture

Five Python modules, one configuration file, one Google Sheet.

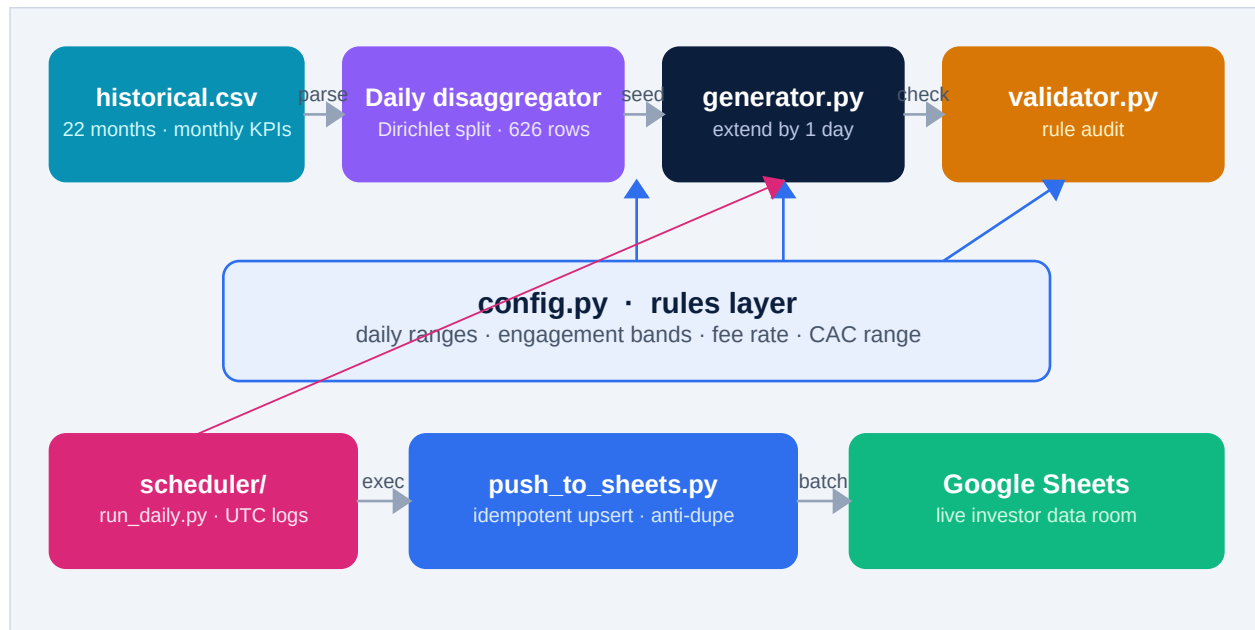


Figure — *historical.csv* → *daily disaggregator* → *generator* → *validator* → *push_to_sheets* → *Google Sheet*. *Rules layer feeds every stage.*

The pipeline is a strict left-to-right chain. The monthly source CSV is read once and deterministically disaggregated to a daily series whose monthly sums match the original exactly (asserted in code, not by convention). The daily **generator** appends a new row for each missing UTC day, seeded with the previous row's cumulative totals. The **validator** then re-checks every synthetic row against the rule layer; if any check fails, the daily run exits non-zero and the Sheets push is skipped.

The **scheduler** (*scheduler/run_daily.py*) ties it all together: it runs the generator and the validator under a single UTC-dated log file, and only invokes the Sheets push if both steps return zero. The Sheets push is itself idempotent — re-running it does not duplicate rows.

SECTION 03

Rules & Dependency Chain

Three logical chains and a transparent revenue formula — every rule lives in `config.py`.

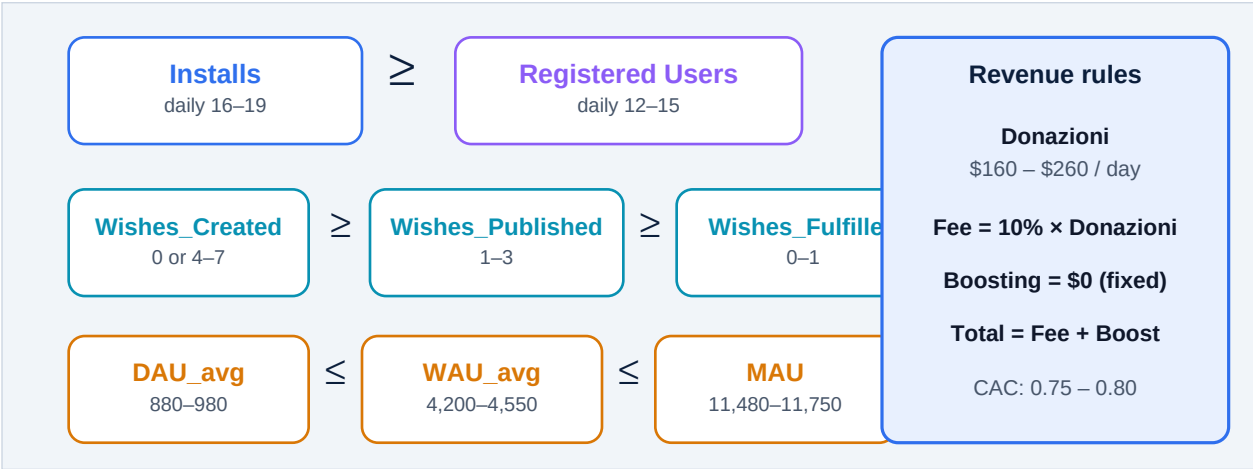


Figure — Three dependency chains plus the revenue rules — all derived from `config.py` at runtime.

The engine enforces three logical chains. **Acquisition:** Installs is always greater than or equal to Registered Users for the same day. **Wishes:** a wish must be created before it can be published, and published before it can be fulfilled. **Engagement:** DAU never exceeds WAU, WAU never exceeds MAU. Each metric also has a configurable daily range; daily values are sampled inside that range.

Revenue is computed by transparent formula rather than learned: daily donation amounts are sampled from a USD range, fee revenue is exactly 10% of donations, boosting revenue is fixed at zero (per current client instruction), and total revenue is the sum. The cumulative "Total Donations" column tracks the running count separately from the daily donation count, so both metrics stay honest.

SECTION 04

Validator Checks

Every synthetic row must pass twelve checks before it can be pushed to the Sheet.

- ✓ No NaN in any key column
- ✓ Cumulative metrics never decrease (monotonic)
- ✓ Wishes_Fulfilled $\leq$ Wishes_Published $\leq$ Wishes_Create
- ✓ Total Donations diff == daily donation count
- ✓ Engagement values inside configured bands
- ✓ Total Revenue == Fee + Boosting
- ✓ Boosting_Revenues_USD always = 0 (fixed)
- ✓ Installs $\geq$ Registered Users
- ✓ Number of Donations within daily range [3 – 9]
- ✓ DAU_avg $\leq$ WAU_avg $\leq$ MAU (engagement order)
- ✓ Fee Revenue == 10% $\times$ Donazioni (rounding-safe)
- ✓ Daily increments inside DAILY_RULES range

Figure — validator.py — the rule audit that runs after every generator pass.

The validator is not optional. It runs after every daily generator pass, and it must return exit code zero or the scheduler skips the Sheets push entirely. If any check fails, the specific offending rows are logged with row index, date, and rule name, so the operator can trace exactly which assumption was violated.

SECTION 05

Technology Stack

Pure Python, pip-installable, no infrastructure beyond a Google service account.

Layer	Technology & Details
Language	Python 3 — pandas + numpy for data shaping; gspread for Sheets I/O
Rule Layer	<i>config.py</i> — daily ranges, engagement bands, donation-fee rate, CAC range, all editable
Disaggregation	<i>historical_data_daily_generator.py</i> — Dirichlet-weighted monthly→daily split with cap; monthly sums asserted to match
Generator	<i>generator.py</i> — daily backfill that obeys DAILY_RULES and the dependency chain
Validator	<i>validator.py</i> — 12 rule checks; non-zero exit on first failure
Sheets I/O	<i>gspread</i> + <i>google.oauth2.service_account</i> — idempotent date-keyed upsert
Scheduling	<i>scheduler/run_daily.py</i> — cron-safe runner with per-day UTC log files; non-zero exit halts downstream
Persistence	Google Sheets is the system of record; CSVs under <i>data/</i> are the local cache

Why this stack. Keeping the engine in pure Python with pip-installable dependencies means it runs anywhere — a developer laptop, a small VPS, a GitHub Actions schedule. Google Sheets as the persistence target lets non-technical stakeholders consume the data room with tools they already use. The rule layer lives in a single Python file so growth assumptions can evolve without touching the engine logic.

SECTION 06

The Challenge

Investor-facing dashboards must look real, behave realistically, and never embarrass the team.

A donation / wishes platform needs a continuously-updating data room that *looks and behaves* like real product analytics, but does not expose actual user data. The client supplied monthly KPI history and a set of growth rules; the job was to turn those into a daily, rules-compliant stream that runs unattended, passes a strict audit, and never duplicates or regresses a row in the live Sheet.

Five constraints made the brief non-trivial: **(1)** every synthetic row had to flag itself, so an analyst can always separate it from history; **(2)** cumulative metrics had to be monotonic — a single regression would break every dashboard chart; **(3)** the daily values had to compose back into the monthly totals the client could verify against their own books; **(4)** the Sheets push had to be idempotent — operators rerun the daily script freely, and duplicates would silently corrupt the data room; and **(5)** the rule layer had to be editable by a non-developer, so growth assumptions could evolve.

Operationally the platform also had to run on commodity infrastructure — no message queues, no data warehouses, no cluster — and emit logs that a non-engineer could read and reason about.

SECTION 07

The Approach

Configuration-driven generation, deterministic disaggregation, idempotent Sheets writes.

Rules in one place

Metric	Daily range / rule
Registered Users	12 – 15 / day (cumulative; never decrease)
Installs	16 – 19 / day $\geq$ Registered Users
Wishes_Created	0 (inactive day) or 4 – 7
Wishes_Published	1 – 3 $\leq$ Wishes_Created
Wishes_Fulfilled	0 – 1 $\leq$ Wishes_Published
Donors	2 – 6 / day
Number of Donations	3 – 9 / day (cumulative running total tracked separately)
Donazioni (USD)	\$160 – \$260 / day
DAU / WAU / MAU	880–980 · 4,200–4,550 · 11,480–11,750
CAC	\$0.75 – \$0.80
Boosting revenue	fixed at \$0
Donation fee rate	10% of Donazioni

Figure — Daily ranges and dependency rules — every value lives in config.py.

Every range, band, and rate the engine touches lives in *config.py*. Growth assumptions evolve by editing numbers in one file; the engine logic never has to change.

SECTION 08

Code Spotlight

Two snippets — the engagement-ordering generator, and the anti-duplicate Sheets upsert.

Engagement values — $DAU \leq WAU \leq MAU$, every time

generator.py

```
# DAU/WAU/MAU are DAILY snapshot levels; always enforce DAU <= WAU <= MAU
def engagement_snapshot_values():
    dau_lo, dau_hi = ENGAGEMENT_RANGES["DAU_avg"]
    wau_lo, wau_hi = ENGAGEMENT_RANGES["WAU_avg"]
    mau_lo, mau_hi = ENGAGEMENT_RANGES["MAU"]

    dau = rand_int(dau_lo, dau_hi)
    wau = rand_int(wau_lo, wau_hi)
    if wau < dau: wau = dau
    mau = rand_int(mau_lo, mau_hi)
    if mau < wau: mau = wau

    # clamp to configured bands, then enforce ordering one more time
    wau = min(max(wau, wau_lo), wau_hi)
    mau = min(max(mau, mau_lo), mau_hi)
    wau = max(wau, dau); mau = max(mau, wau)
    return int(dau), int(wau), int(mau)
```

Idempotent Sheets upsert — re-run safely, no duplicates

push_to_sheets.py

```
# Hard guard against re-appending existing dates (the 13 Jan duplication fix)
max_sheet_date = max(sheet_dates) if sheet_dates else None
updates, append_rows = [], []

for row_vals in csv_rows:
    d = str(row_vals[date_idx]).strip()
    if d in date_to_rownum:
        # date already exists -- UPDATE that row
        updates.append((date_to_rownum[d], row_vals))
    else:
        # append ONLY if strictly newer than every row in the sheet
        if max_sheet_date is None or d > max_sheet_date:
            append_rows.append(row_vals)

if updates: ws.batch_update(updates_payload)
if append_rows: ws.append_rows(append_rows, value_input_option="RAW")
```

SECTION 09

Outcome & Engagement

A self-refreshing, rules-compliant data room that operators trust and stakeholders can read.

The Synthetic Data Engine delivers a Google Sheet that updates itself, behaves like real product analytics, and survives investor scrutiny. Every synthetic row is flagged, validated, and explainable; every rule lives in one file that a non-developer can adjust; every Sheets push is idempotent so operators can rerun freely. When something goes wrong — a malformed row, a regressing cumulative, an engagement ordering violation — the validator stops the run before any of it reaches the live data room.

Architecturally the system is built to evolve. New metrics are a config entry plus an extension to the rule chain; new validation checks are a function added to *validator.py*; swapping Google Sheets for a database is a single push-module change. The engine layer never moves — only the rule layer and the sink layer do.

- › **Configuration-driven generation** — every range, band, and rate lives in *config.py*; growth assumptions evolve without code changes.
- › **Math-guaranteed disaggregation** — Dirichlet weights split monthly totals into capped daily values; round-trip equality asserted in code.
- › **12-check validator** — cumulative monotonicity, engagement ordering, dependency chains, 10% fee correctness, daily-increment ranges.
- › **Idempotent Sheets upsert** — date-keyed batch updates for existing rows; hard guard against duplicate appends, even on rerun.
- › **Unattended operation** — cron-safe scheduler runs generator + validator daily under one UTC-dated log file; failure codes stop the downstream push.
- › **Honest synthetic data** — every generator-produced row is flagged *is_synthetic=True* so analysts can always tell history from generation.

LET'S TALK

Mudasir Shafiq

Full-stack & data engineer

Email chmudasir.fr@gmail.com

Stacks Python • pandas • Airflow • BigQuery • React • Spring Boot